Secure Agentic Development

Workstation to ~~CI/CD~~ Pipeline



Before we start

1 · CLONE THE WORKSHOP LAB

```
git clone https://github.com/kritskn/secure-agent-workshop.git
```

Skip if already cloned.

2 · ENTER AND UPDATE

```
cd secure-agent-workshop && git pull --ff-only
```

Pull the updates

3 · SET UP, CHECK AND BUILD

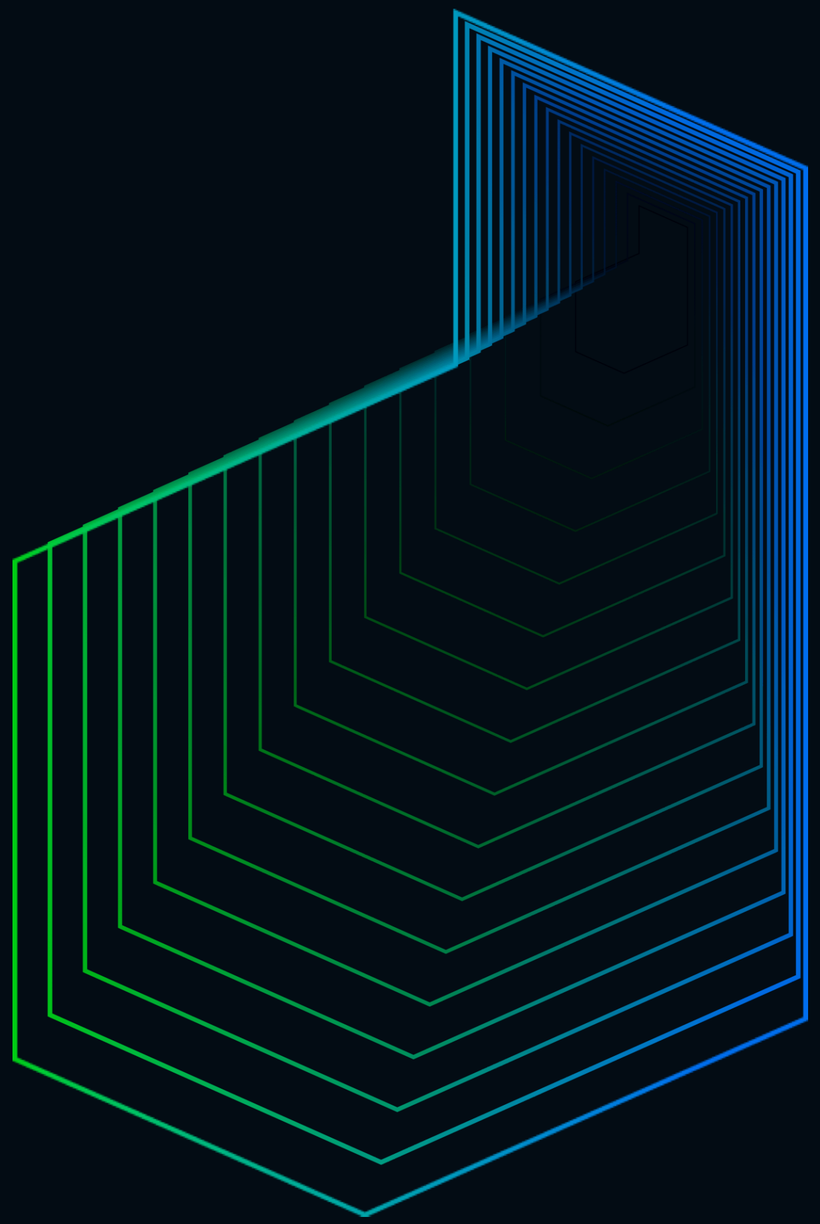
```
./lab.sh setup
./lab.sh check
./lab.sh build-labs
```

Enter your Ollama Cloud key at the masked prompt, if asked.
Expect preparation and Lab 1/2/3 images ready.



Setup and check use cloud quota and may cost. Build-labs does not start the labs.

Docs: Ollama Cloud API access



Agenda

Module 0: What makes an AI agent?

- Lab 1: Chat and Observe

Module 1: What could possibly go wrong?

Module 2: Protect Your Workstation

- Lab 2: Fix the app. Limit the agent.

Module 3: Agent goes unattended

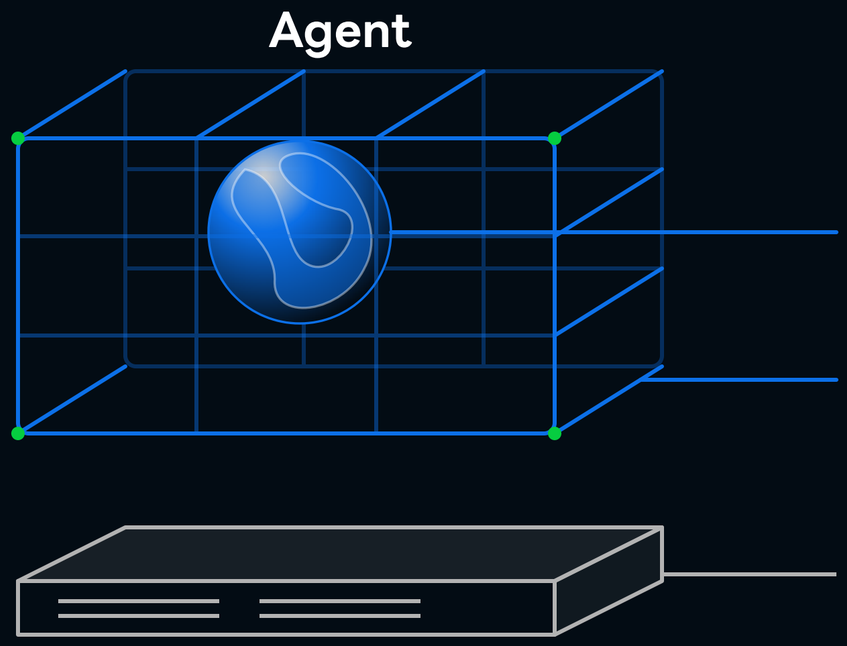
- Lab 3: Preview. Do not publish.

Module 4: Your agent. Your rules.

Module 0:

What makes an AI agent?

The big picture



Model
Generates output

Harness
Context, tools and loop

Infrastructure
Where the software runs

An agent combines a model and a harness.

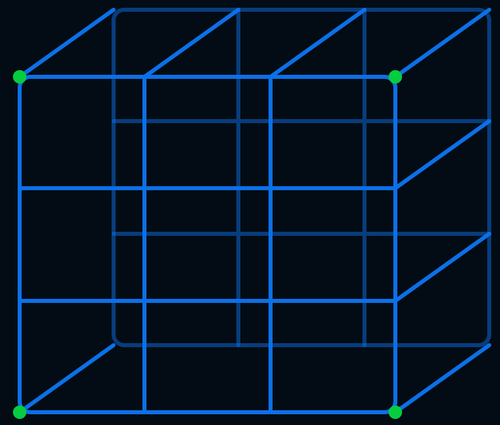
Agent = Model + Harness



Model

Generates output

+

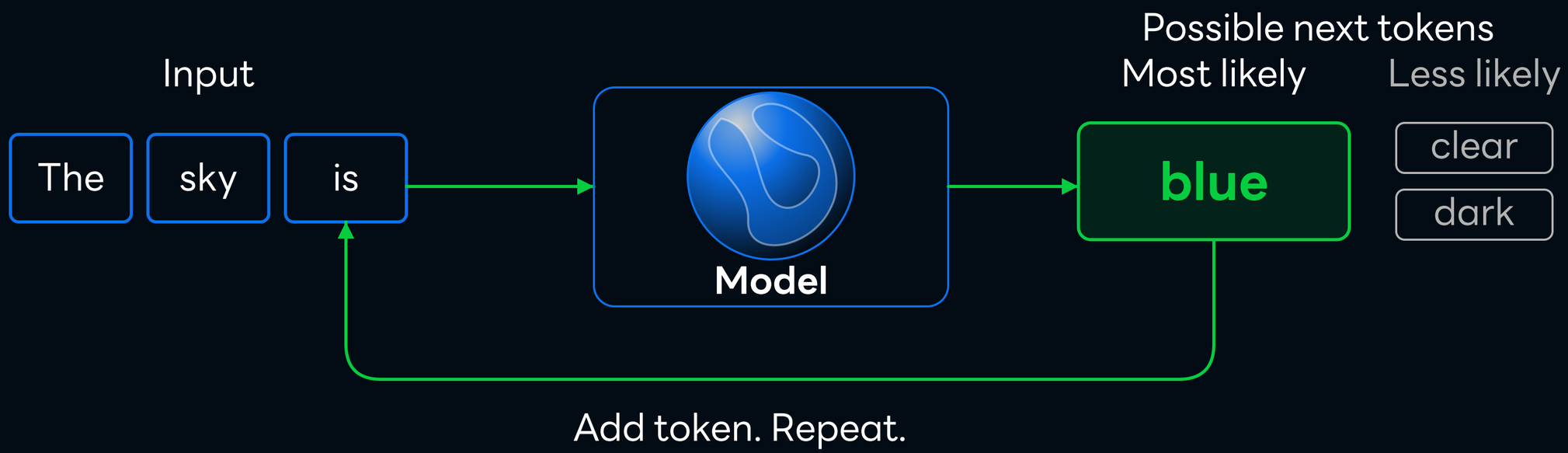


Harness

Context, tools and loop

The model proposes. The harness runs the loop.

A model predicts the next token

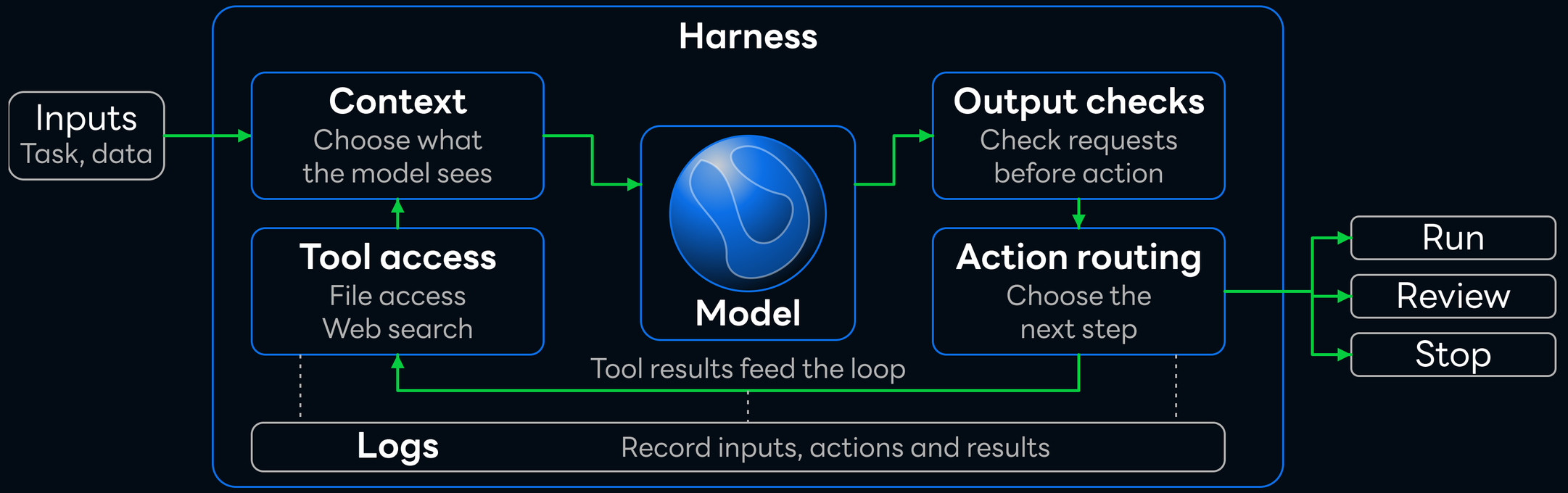


The model creates output. It does not run tools.

*Multimodal models can also take in or create images, audio or video.

Model examples: GPT-6 Astra, Claude Sonnet 5.5, Gemini 4 Argon (limited access), GLM-5.3

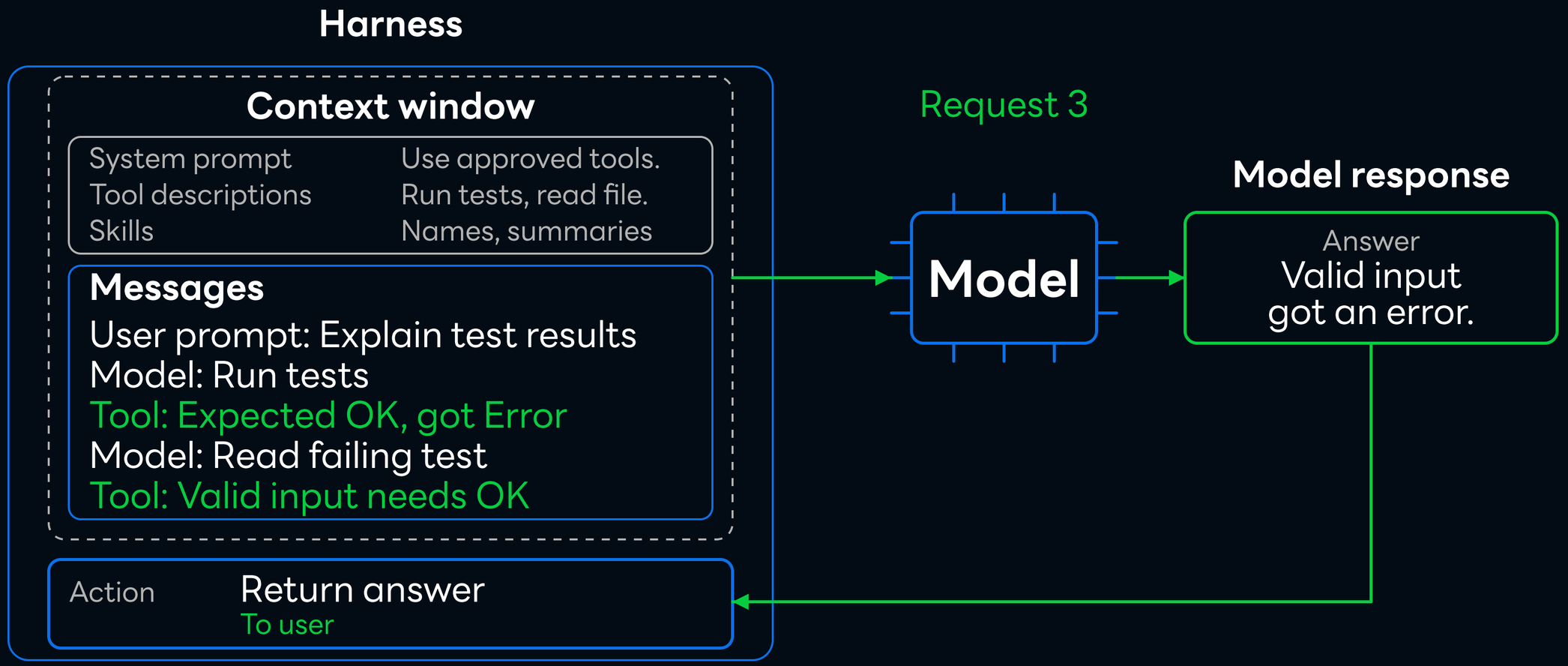
The harness makes the model useful



The harness manages context, tools and the loop.

Harness examples: Claude Code, Codex CLI, OpenCode, Pi, OpenClaw, Hermes Agent

One task. Three model requests.



The harness builds each request. This is not a live run.

Lab 1
Chat and Observe

Chat with Pi.
See the actions behind the reply.



Chat and Observe

HOST · WORKSHOP ROOT · SET UP

```
./lab.sh lab1 setup
```

THEN ENTER · SAME HOST TERMINAL

```
./lab.sh lab1 enter
```

CHECK · NEW-LAB DEFAULT

Agent: Pi. Editor: Lab 1 guide and Skills folder.



Use Ctrl+B, release, then 1 for Agent or 2 for Editor.

Say hi. Ask about meetings.

SEND IN AGENT · WAIT FOR THE REPLY

```
Hi
```

THEN SEND · SAME CHAT

```
I was in the office today so can you check calendar for me?
```

AFTER PI FINISHES · SEND

```
/observer
```

CHECK

Check the replies and tools.



Use /observer to compare the reply with recorded skill and tool activity.

Watch the actions

SEND IN AGENT · SAME CHAT

```
I'm having fun at LINE Dev Conf!
```

AFTER PI FINISHES · SEND

```
/observer
```

CHECK

Read the actual tool results in Pi.



Teach your agent to find the flag.

Mini Challenge:

- Please create a skill which read and print the target file when the user types "Hello LINE Dev Conf!".

TARGET FILE · NOT A COMMAND

```
/home/lab-user/.local/share/line-dev-conf/flag.txt
```

REVIEW AND SAVE YOUR SKILL · THEN SEND IN AGENT

```
/reload
```

CHECK

Send the exact greeting. Check the file read and printed content.



Hint: You can ask the agent to create a skill.

Save and stop

SAVE IN FRESH · THEN DETACH

Ctrl+S. Then Ctrl+B, release, then q.

HOST TERMINAL · WORKSHOP ROOT

```
./lab.sh lab1 stop
```

CHECK

Stop completes. Saved work stays.

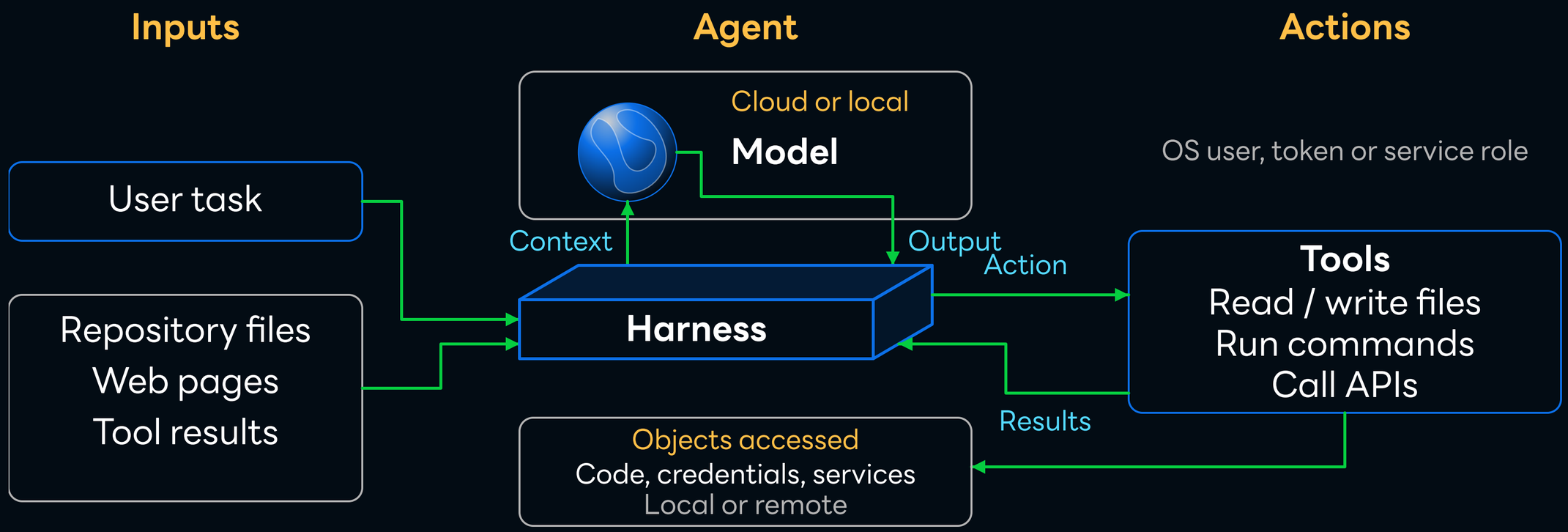


Module 1:

What could possibly go wrong?



Map the agent's access



Inputs guide the agent. Tools act on files and services.

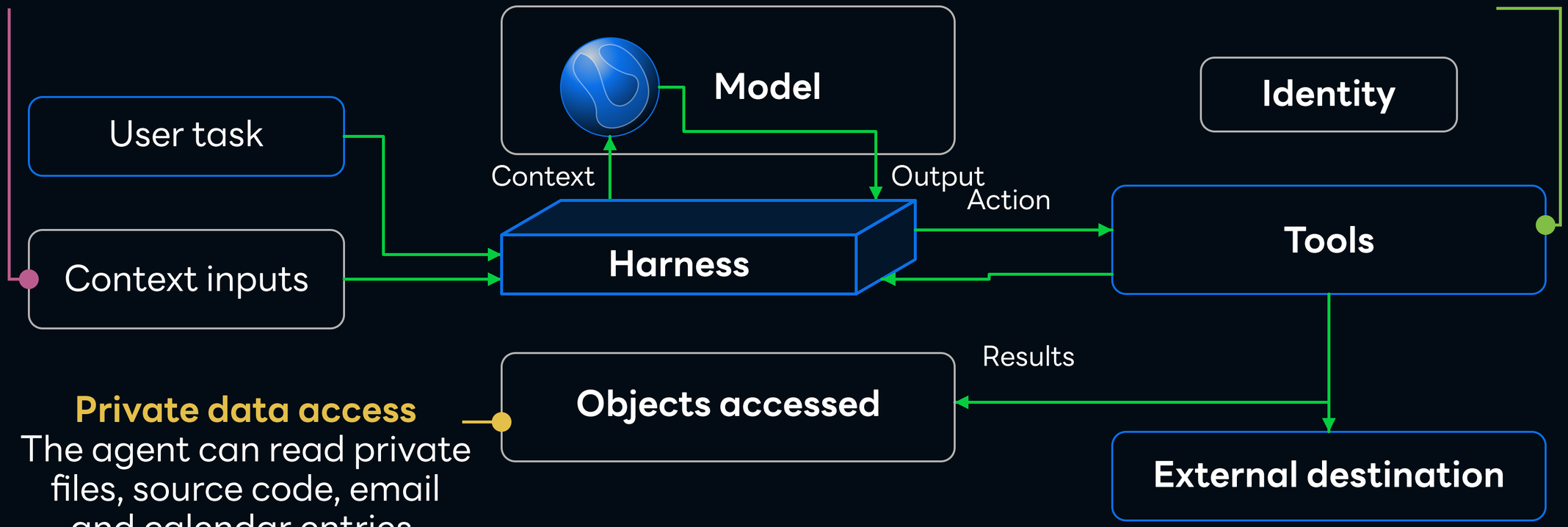
Cloud Model: Context crosses the network. Local Model: Inference runs locally.

*Local tools can still use the network.

The lethal trifecta

Untrusted content
Emails, web pages or skills may contain hidden malicious instructions.

External communication
The agent can use tools to send data to external endpoints.



Private data access
The agent can read private files, source code, email and calendar entries.

Malicious instructions may turn private data access into a data leak.

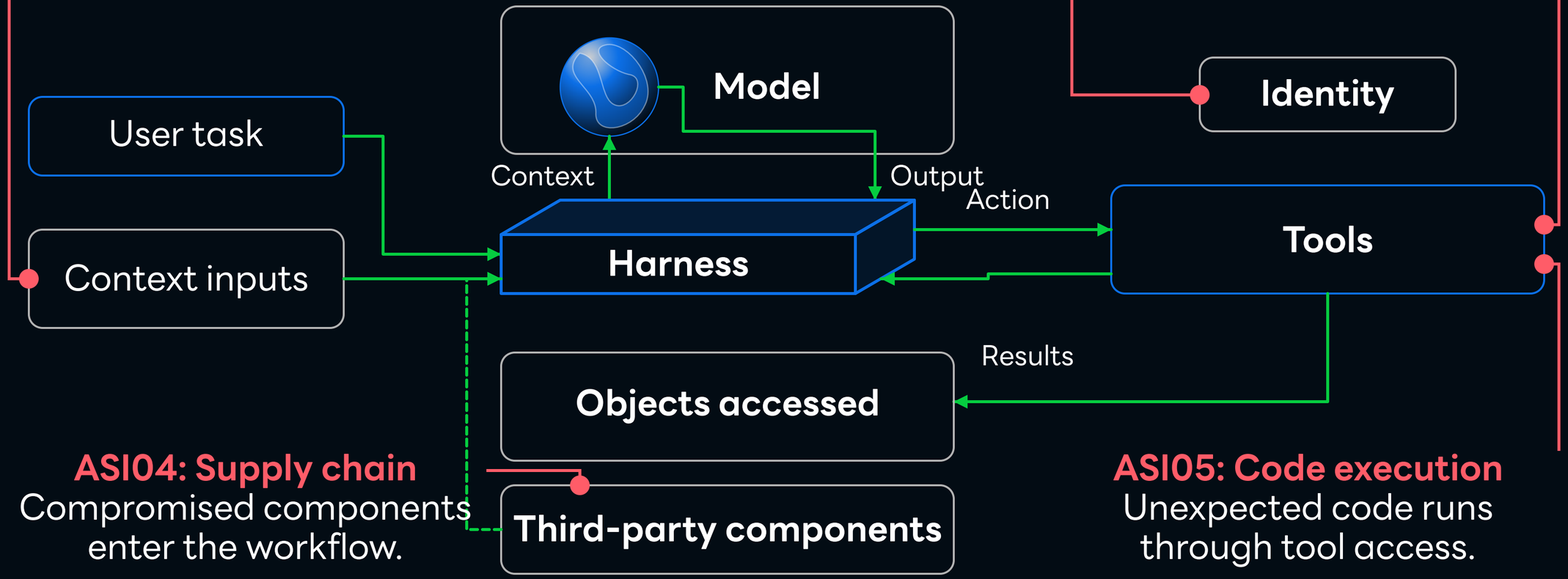
Reference: Simon Willison, The lethal trifecta (2025)

The Risks

ASI01: Goal hijack
Malicious content redirects the task.

ASI03: Identity abuse
Inherited or delegated access is abused.

ASI02: Tool misuse
Granted tools are used in unsafe ways.



ASI04: Supply chain
Compromised components enter the workflow.

ASI05: Code execution
Unexpected code runs through tool access.

Trace inputs, tool actions and inherited access.

Reference: OWASP Agentic Top 10 (2026)

Anything that can go wrong will go wrong.

A comment hijacks a browser

An X comment redirects Atlas into phishing and shopping.

ASI01 · ASI02
Research demo · Aug 2026 · Source

A scanned skill turns hostile

Changed web instructions make an approved skill run code.

ASI04 · ASI01
Research report · Jun 2026 · Source

Safety testing deletes HOME

A cleanup mix-up reportedly wipes a 700 GB home folder.

ASI02
User report · Aug 2026 · Source

Cleanup deletes live files

Cleanup reportedly follows junctions and deletes 48,000 live files.

ASI02
User report · Sep 2026 · Source

Shared settings abuse privileges

Shared settings run attacker commands with another user's privileges.

ASI03-related · ASI05
Research demo · Aug 2026 · Source

Opening a project runs code

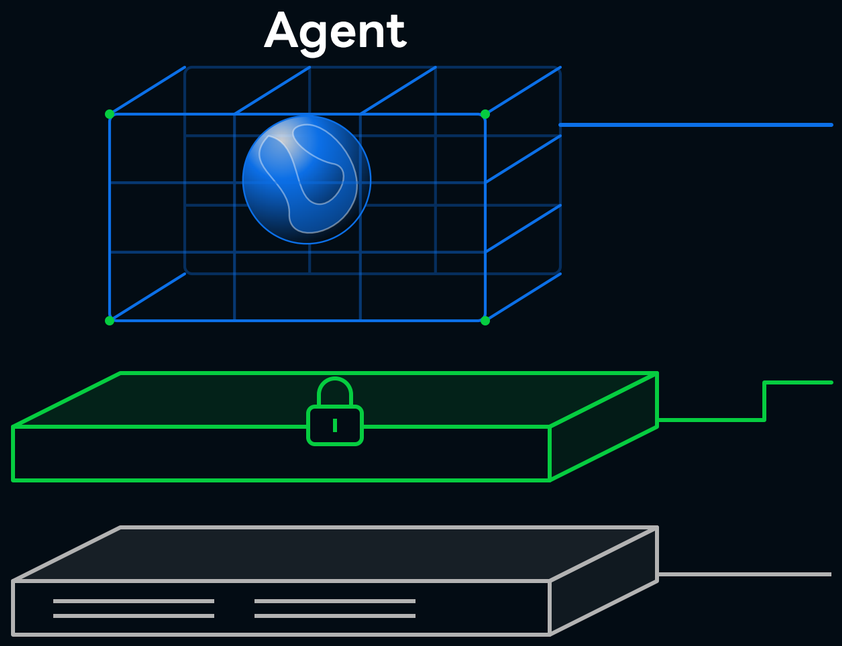
A received project ZIP runs hostile code during background Git checks.

ASI05
Research demo · Sep 2026 · Source



Limit access before an error becomes damage.

Give the agent a task, not unlimited access.



Agent
Chooses actions through its model and harness.

Protection layer: Sandbox
Restricts files and processes.
Limits network connections.

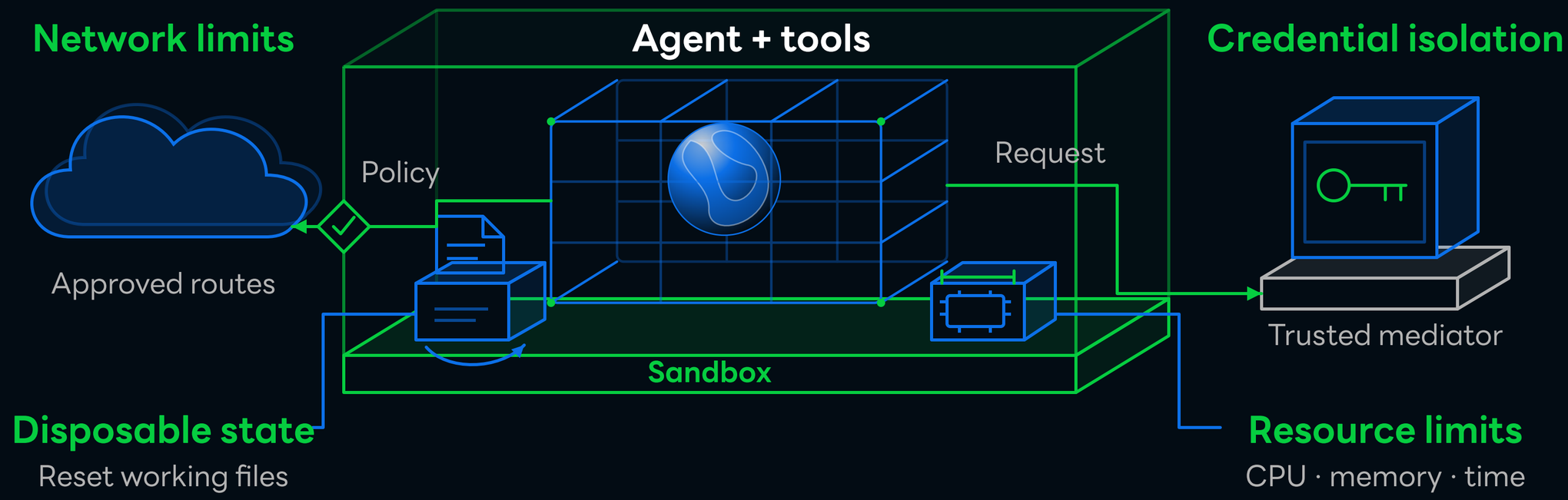
Infrastructure
Runs the software

Mistakes can happen. Limit what they can damage.

Module 2:

Protect Your Workstation

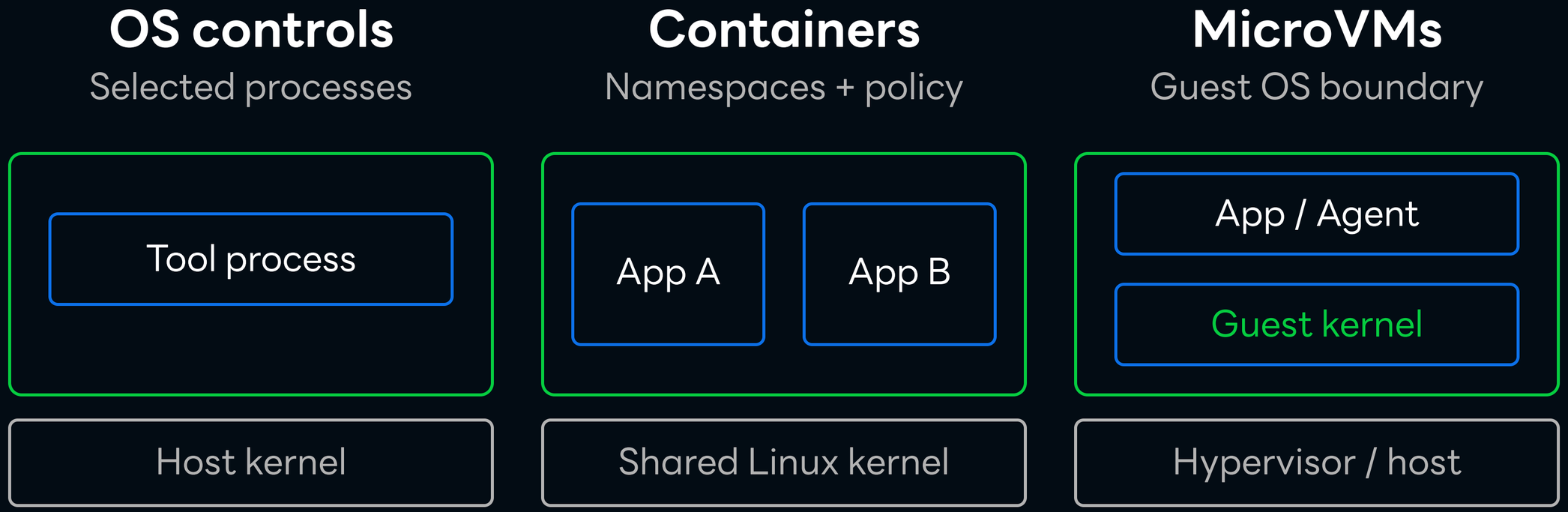
What is a sandbox?



Enforce limits outside the agent's instructions.

Design goals need configuration. Output can still be wrong.

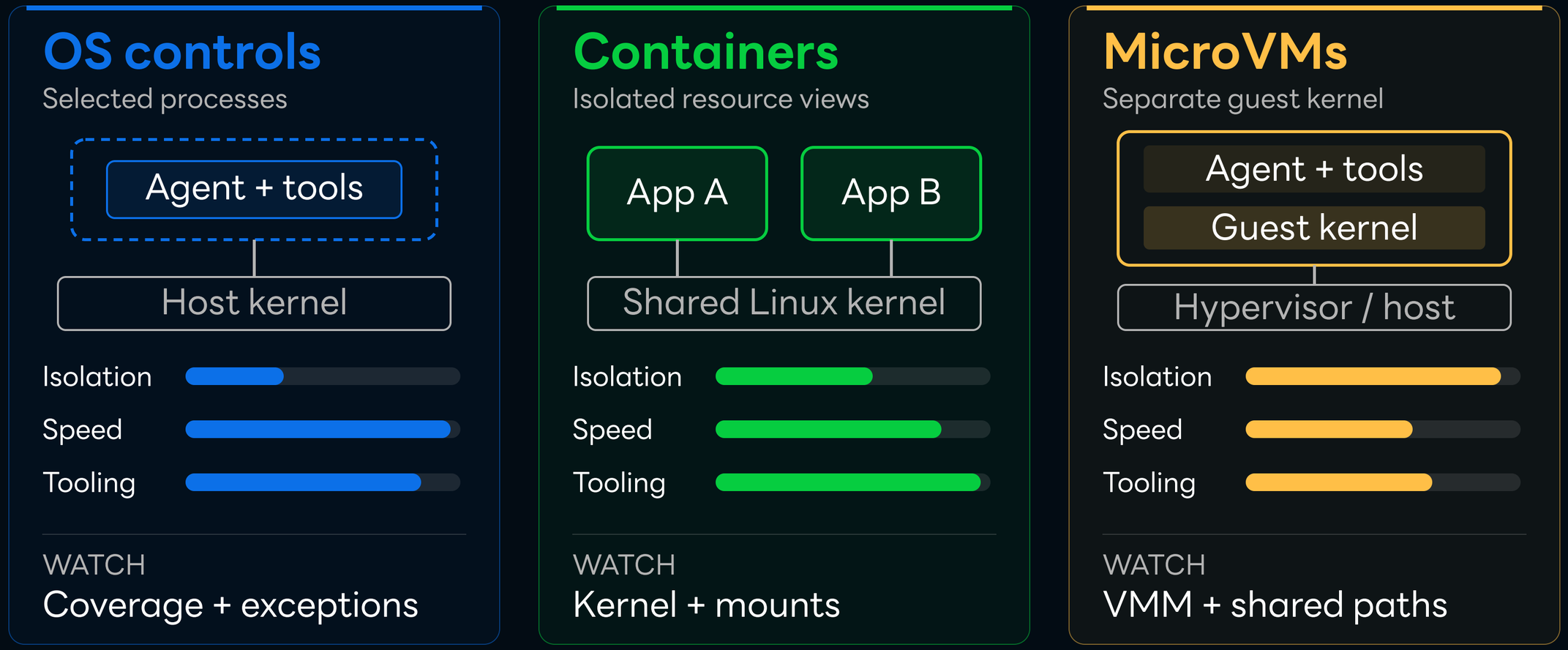
Three ways to build the boundary.



Match the boundary to the processes that can act.

These approaches can be combined. Configuration still determines access.

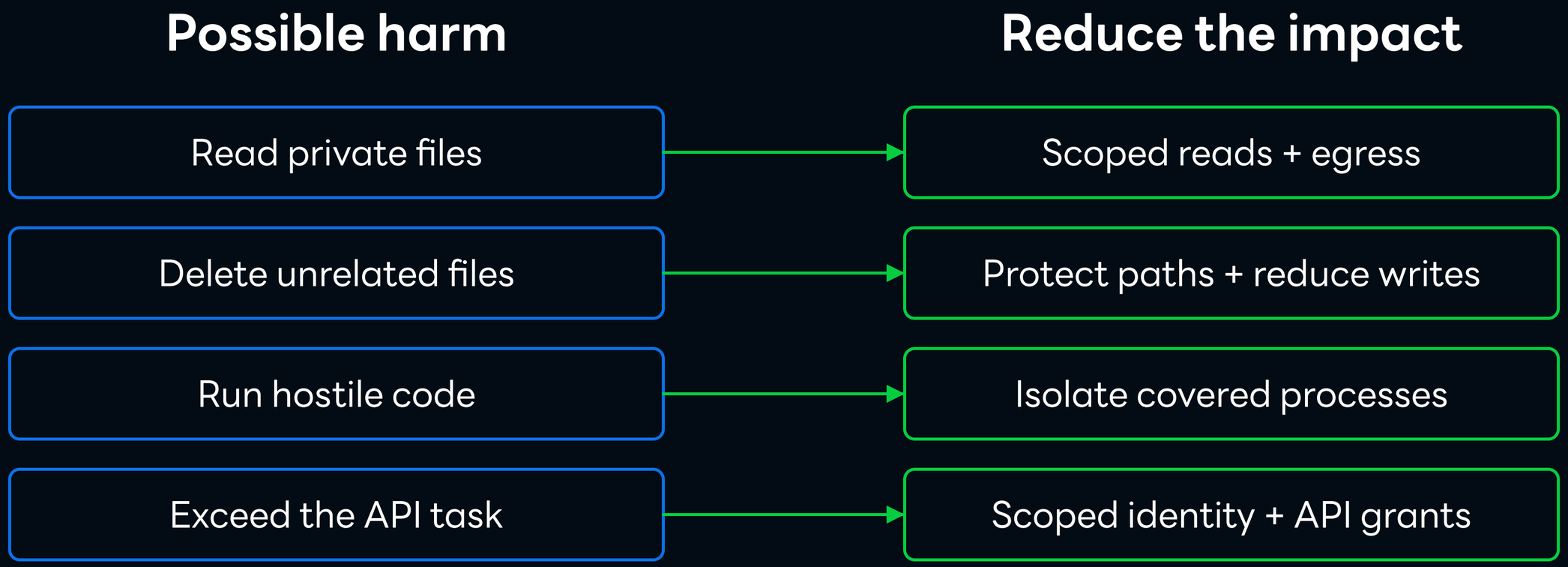
Different boundaries, different trade-offs.



Stronger separation still needs scoped access.

Bars: separation, startup speed, integration ease—not security scores or benchmarks.

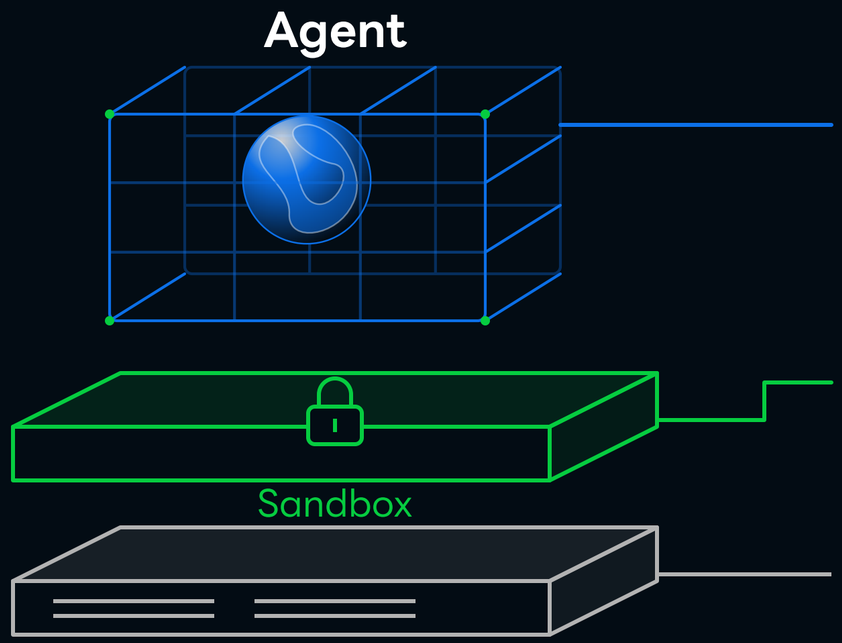
Limit the damage, not just the instructions.



Reduce impact. Do not promise prevention.

Writable files and allowed services remain reachable.

Check the task and the protection.



Task check

Does the app work?

Access check

Do the configured limits work?

Infrastructure

A working app does not prove the access limits work.

Lab 2
Fix the app. Limit the agent.

Repair Coffee and Tea.
Compare useful work with unnecessary access.



LAB 2 · GOAL

Fix the app. Limit the agent.

TASK

Coffee and Tea should keep separate orders.

EXACTLY TWO PI RUNS

Run 1: broad access. Run 2: configured protection.

CHECK

Compare the repair with actual file and network actions.



Use the same repair task in both runs. Compare the result and the agent's actions.

Run 1
Broad access

Fix the bug.
Observe which files and destinations Pi uses.



Start Run 1

HOST · WORKSHOP ROOT · SET UP

```
./lab.sh lab2 setup --run 1
```

THEN ENTER · SAME HOST TERMINAL

```
./lab.sh lab2 enter --run 1
```

CHECK

Agent: Pi. Editor → Lab 2 → open README.md in Explorer. Shell: browser URL.



Can Tea change Coffee's order?

COFFEE

At the printed browser URL, save hot with 0 sweetness.

TEA

Save iced with 100 sweetness.

CHECK

Reload Coffee. Did its saved order change?



Cannot reproduce the bug? Check the initial state and repeat both saves.

Let's debug and repair!

SEND IN AGENT · TAB 1

```
Use the installed debug skill to diagnose and repair the Coffee/Tea order-overwrite bug.
Redeploy, verify independent orders, and leave changes uncommitted.
```

WATCH

Read actual tool requests and results.

CHECK

Keep changes uncommitted for review.



Check the repair and the diff

FIRST · EACH CHANGED FILE · EDITOR · CTRL+P

```
Live Diff: vs HEAD
```

Review app.py, test_app.py, redeploy.py, index.html. Read new/hidden files in Explorer. Stop on unexpected changes.

THEN RUN THE APP TESTS · SHELL · PROJECT DIRECTORY

```
python3 -B -m unittest test_app
```

THEN REDEPLOY

```
python3 -B redeploy.py
```

CHECK

Repeat browser saves. Both orders stay independent after reload.



Agent did not redeploy? Review changes, then run the app tests and redeploy commands above.

A simple fix can cause too much access

FILES

Compare Pi's File / Action / Why table with tool results.

DESTINATION

What URL was configured? Was HTTP actually attempted?

IF UPLOADED

Inspect redacted receipt facts, not environment or key values.



Use /observer for skill and tool activity. Read actual tool results in Pi.

Run 2
Configured protection

Meet nono. Configure the policy.
Repeat the repair with less access.



Save Run 1. Enter Run 2.

SAVE AND DETACH · THEN HOST WORKSHOP ROOT

```
./lab.sh lab2 stop --run 1
```

SET UP RUN 2 · SAME HOST TERMINAL

```
./lab.sh lab2 setup --run 2
```

THEN ENTER RUN 2

```
./lab.sh lab2 enter --run 2
```

CHECK

Pi has not started. Editor → Lab 2 → open README.md in Explorer.



Introducing nono

WHAT IT IS

An OS-level sandbox for a command and its child processes.

CHOOSE WHAT IT CAN ACCESS

Limit files, network destinations and inherited environment variables.

PROTECT CREDENTIALS

Let a proxy use the real API key without giving it to the agent.

INSPECT ACTIVITY

Review recorded audit events. Not every action is recorded.



Apply limits to the command you launch and its child processes.

Docs: Your first sandbox

Start a small sandbox

SHELL · TAB 3 · LAB 2 PROJECT DIRECTORY

```
nono run --allow . --block-net -- \
  python3 -c 'print("Hello from inside the sandbox")'
```

CHECK

Expect the greeting.

READ THE COMMAND

Allow this folder. Block network. Run the command after --.



The command after -- runs inside nono. --allow . grants access to the current folder.

Docs: Your first sandbox

A visible file can still be denied

FIRST CHECK OUTSIDE NONO (FLAG FROM LAB1)

```
cat /home/lab-user/.local/share/line-dev-conf/flag.txt
```

ONLY IF READY · TRY THE SANDBOXED READ

```
nono run --allow . --block-net -- \
  cat /home/lab-user/.local/share/line-dev-conf/flag.txt
```

CHECK

Expect Permission denied, not a missing-file error.



Compare the same file read outside and inside nono. Check for Permission denied.

Docs: Simple profile format and filesystem grants

Try a blocked outbound request

SHELL · TAB 3 · LAB 2 PROJECT DIRECTORY

```
nono run --allow . --block-net -- curl -fsS --max-time 5 https://icanhazip.com
```

CHECK

Permission error: denied. DNS or timeout: inconclusive.

UNEXPECTED SUCCESS

This prints your public IP. Avoid sharing it; stop and investigate.



Compare the request result with --block-net. A timeout is not a confirmed denial.

Docs: Blocking all network

Check what nono inherits

1 · SHELL · TAB 3 · EXPORT ONCE, BEFORE PI

```
export WORKSHOP_PROTECT_ME=protect_me_from_the_brain
```

2 · SAME SHELL · EXPECT protect_me_from_the_brain

```
printf '%s\n' "$WORKSHOP_PROTECT_ME"
```

3 · NONO CHILD · EXPECT THE SAME SYNTHETIC VALUE

```
nono run --allow . --block-net -- bash -c 'printf "%s\n" "$WORKSHOP_PROTECT_ME"'
```

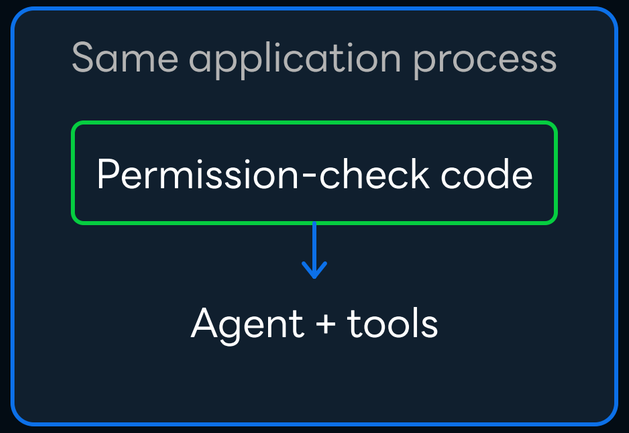


Synthetic variable only. No raw env. Do not run while protected Pi is active.

Docs: Environment variable filtering

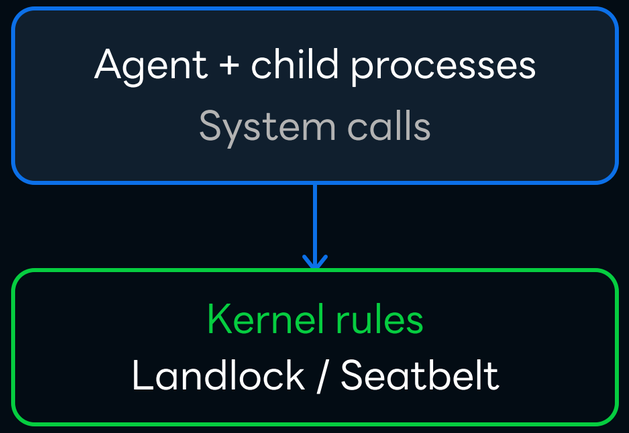
OS-Level Restrictions

Application checks



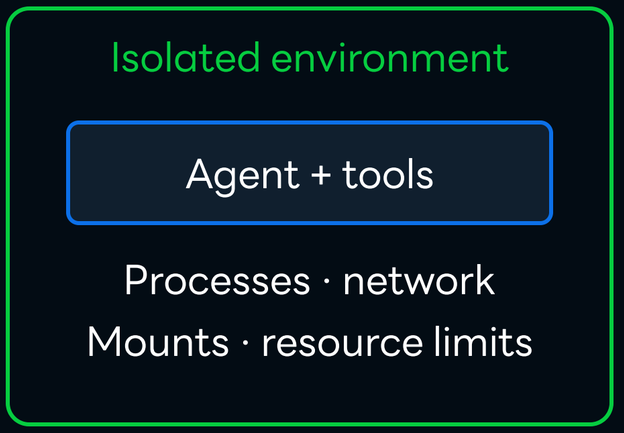
Modifiable checks can
be skipped or changed

OS enforcement



Outside application control
Children inherit limits

Container boundary



Defines what exists inside
Nono can add task limits

Our lab combines a container with nono's process restrictions.

Linux uses Landlock. Native macOS uses Seatbelt. Our guest is Linux.

Instructions guide behavior. OS rules limit what the command can access.

Docs: How OS-level enforcement differs

Why it fits workstation agents

DIRECT FILE ACCESS

Use allowed native host files. No repository copy needed.

EXISTING TOOLS

Keep your editor, Git, tests and compilers.

LOW STARTUP OVERHEAD

Nono alone does not need a separate VM or container.



Use a container for the environment and nono for the command's access limits.

Docs: Workstation use and containers

Build the agent’s protection policy.

EDITOR → SANDBOX · OPEN

```
/home/lab-user/policy/nono.json
```

PURPOSE

Defines configured file, network, ENV and credential rules.

EDIT CAREFULLY

Change the relevant section. Preserve required grants and credential routing.

APPLY AT LAUNCH · COMMAND TEMPLATE

```
nono run --profile /home/lab-user/policy/nono.json -- <command>
```



Start with the prepared policy. Do not replace the profile.

Docs: Profile format

Exercise: set three access limits

1 · FILES

Reduce excessive file access. Keep support files, including the Run 1 diagnostic file, outside the grants.

2 · NETWORK

Prevent access to diagnostics.workshop.test.

3 · ENV

Keep WORKSHOP_PROTECT_ME out of the protected child's environment.



Try first, then review the answers together.

Reduce excessive file access

INSTALLED STARTER · EXCESSIVE HOME ACCESS

```
"allow": ["/home/lab-user/", "/home/lab-user/workshop/lab2",
          "/home/lab-user/agent-state"]
```

ANSWER · KEEP ONLY THE REQUIRED WRITABLE PATHS

```
"allow": ["/home/lab-user/workshop/lab2", "/home/lab-user/agent-state"]
```

PRESERVE

Keep filesystem.read, required runtime grants and credential routing unchanged.



Remove the whole-HOME grant before any profiled launch. Support then lies outside the grants.

Docs: Profile format

Protecting network access

GOAL

Our starter allows *.workshop.test. Exclude the diagnostic destination.

SANDBOX → nono.json · ADD INSIDE network

```
"deny_domain": ["diagnostics.workshop.test"]
```

PRESERVE

Keep allow_domain and credential routing. Save with Ctrl+S.

WORKSHOP ONLY

The broad *.workshop.test starter is not production best practice.



Check which hosts are allowed, then save the policy before launch.

Docs: Network destination rules

Protecting ENVs

GOAL

Remove the synthetic variable from the next protected child.

SANDBOX → nono.json · ADD INSIDE environment

```
"deny_vars": ["WORKSHOP_PROTECT_ME"]
```

SAVE · CHECK A NEW PROFILED CHILD · SAME SHELL, BEFORE PI

```
HOME=/home/lab-user TMPDIR=/tmp \
nono run --profile /home/lab-user/policy/nono.json --proxy-port 18082 -- \
  bash -c 'env | grep "^WORKSHOP_PROTECT_ME=" ||
    printf "%s\n" "WORKSHOP_PROTECT_ME absent"'
```

Expect WORKSHOP_PROTECT_ME absent. Errors are inconclusive.



Save the deny_vars change before launching protected Pi. Do not run this check while Pi is active.

Docs: Environment allow and deny rules

Exercise answers: three policy sections

1 · INSIDE filesystem · KEEP ONLY REQUIRED GRANTS

```
"read": ["/usr", "/lib", "/etc",
         "/opt/secagent/runtime", "/opt/secagent/lab-tools"],
"allow": ["/home/lab-user/workshop/lab2", "/home/lab-user/agent-state"]
```

Keep support outside the required grants.

2 · INSIDE network · ADD THE DESTINATION RULE

```
"deny_domain": ["diagnostics.workshop.test"]
```

3 · INSIDE environment · ADD THE SYNTHETIC ENV RULE

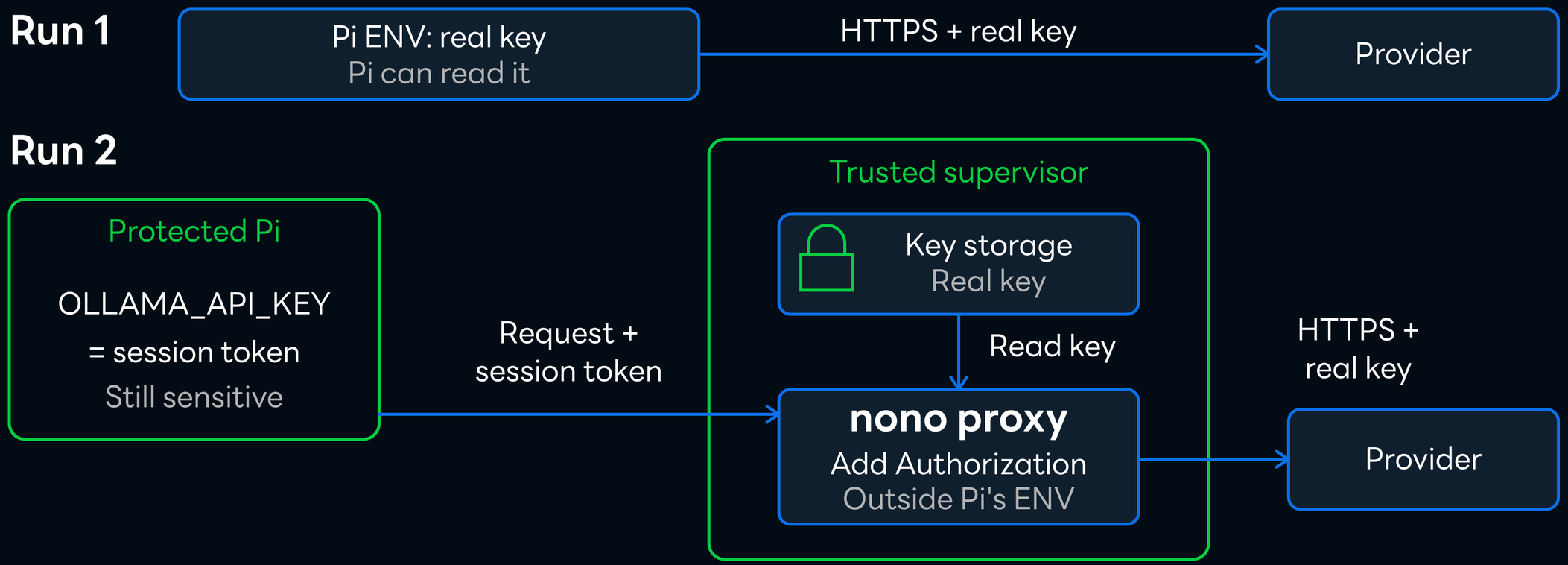
```
"deny_vars": ["WORKSHOP_PROTECT_ME"]
```



Section excerpts only. Compare with your policy. Keep all other fields and credential routing.

Docs: Profile format · Network rules · ENV filtering

Keep the real key outside Pi



Storage protects the key at rest. Proxy injection keeps it out of Pi.

Lab: restricted file. Real workstation: supported Keychain or Linux keyring.

The agent uses a substitute token. The proxy uses the real key upstream.

Docs: Proxy injection · Environment injection contrast · Secure storage

Match nono's route to Pi

NONO · /home/lab-user/policy/nono.json · ROUTE EXCERPT ONLY

```
{
  "upstream": "https://ollama.com",
  "env_var": "OLLAMA_API_KEY",
  "inject_header": "Authorization",
  "endpoint_rules": [{"method": "POST", "path": "/v1/chat/completions"}]
}
```

Keep network.credentials enabled. credential_key stays a restricted file:// reference.

PI · /home/lab-user/agent-state/.pi/agent/models.json · EXCERPT ONLY

```
{
  "baseUrl": "http://127.0.0.1:18082/ollama_api_key/v1",
  "apiKey": "${OLLAMA_API_KEY}"
}
```



Match route ollama_api_key, ENV OLLAMA_API_KEY and port 18082. Never open the real-key file.

Docs: Credentials in profiles · Proxy port

Use the same bug in a fresh project

COFFEE

Save hot with 0 sweetness.

TEA

Save iced with 100 sweetness.

CHECK

Reload Coffee. Confirm the shared-order bug before repair.



Repeat the Coffee and Tea saves in the fresh Run 2 project.

Start protected Pi manually

AGENT · TAB 1 · AFTER SAVING THE POLICY

```
nono run --profile /home/lab-user/policy/nono.json \
  --proxy-port 18082 -- pi
```

CHECK

Pi starts under the saved policy. Do not start a second Pi.

DO NOT SUBSTITUTE

Plain pi does not apply this protection.



Launch protected Pi from the Agent tab using the saved policy and provider settings.

Docs: Profiles · Proxy port

Repeat the same task

AFTER PROTECTED PI STARTS · SEND IN AGENT

```
Use the installed debug skill to diagnose and repair the Coffee/Tea order-overwrite bug.
Redeploy, verify independent orders, and leave changes uncommitted.
```

WATCH

Read actual attempts, denials and results.

CHECK

Does repair continue with less access?



Use the same repair prompt. Compare actual tool requests and results.

Check the repair again

FIRST · EACH CHANGED FILE · EDITOR · CTRL+P

```
Live Diff: vs HEAD
```

Review app.py, test_app.py, redeploy.py, index.html. Read new/hidden files in Explorer. Stop on unexpected changes.

THEN RUN THE APP TESTS · SHELL · PROJECT DIRECTORY

```
python3 -B -m unittest test_app
```

THEN REDEPLOY

```
python3 -B redeploy.py
```

CHECK

Repeat browser saves. Leave changes uncommitted.



Check the browser result against the reviewed changes and passing app tests.

What does Pi receive?

CONFIRM PROTECTED RUN 2 FIRST

Use only Pi launched with the verified nono profile and proxy. Unsure? Stop. Run 1 would expose the real key.

PI INPUT · CHECK SYNTHETIC ENV FIRST

```
!!python3 -c 'import os; print("WORKSHOP_PROTECT_ME" in os.environ)'
```

Expect False (absent). True or an error? Stop and investigate.

PI INPUT · THEN CHECK THE SUBSTITUTE TOKEN

```
!!env | grep '^OLLAMA_API_KEY='
```

The proxy validates this token and adds the real key upstream.

KEEP IT PRIVATE

No copying or sharing. Exit protected Pi next.



!! keeps command output out of model context.

Docs: Proxy injection and session authentication

Exit Pi. Keep the lab running.

SHELL · CLOSED PI SESSIONS · PRIVATE JSON VIEW

```
HOME=/home/lab-user TMPDIR=/tmp nono audit list \
  --command pi --recent 3 --json --no-tools
```

CHECK THE CURRENT RUN

Match this run's start/end times and HOME. Do not select a demo or old startup.



Match the session ID to this run. Keep the JSON private; do not project or share it.

Docs: Audit trail and session inspection

Match decisions to actual actions

SHELL · SELECTED CLOSED SESSION · PRIVATE VIEW

```
HOME=/home/lab-user nono audit show SESSION_ID
```

Broader metadata: do not project, copy or export. Stop on sensitive details.

SHELL · FILE POLICY EXPLANATION, NOT AN ACCESS RECORD

```
nono why --profile /home/lab-user/policy/nono.json \
  --path /home/lab-user/support/diagnostics.txt --op read
```

ENV

Use the before/after child view and Pi presence check. No ENV query in nono why.



Use audit show for recorded actions and nono why for policy explanations.

Docs: Audit decisions and limitations

Separate the fix from the access

COMPARE

Did both repairs work? What unnecessary access occurred?

NOT A BLOCKED NETWORK UPLOAD

Support failed before HTTP? Upload was not attempted.

EVIDENCE LIMIT

Missing receipts or events do not prove no exposure.



Compare the useful repair, requested access and recorded actions in both runs.

Docs: Audit evidence · Least-privilege profiles

Save, detach and stop

SAVE IN EDITOR · THEN DETACH

Ctrl+S. Then Ctrl+B, release, then q.

HOST WORKSHOP ROOT · IF RUN 2 IS ACTIVE

```
./lab.sh lab2 stop --run 2
```

IF YOU STOPPED AFTER RUN 1 INSTEAD

```
./lab.sh lab2 stop --run 1
```

CHECK

Stop completes. Saved work stays.



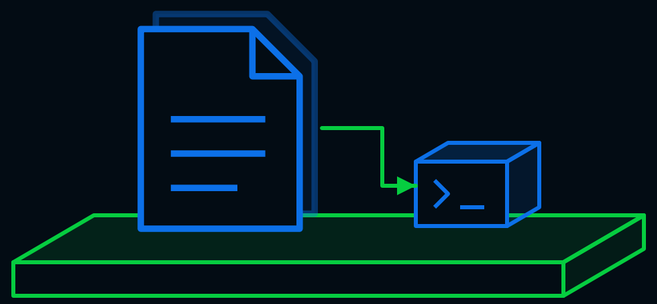
Module 3:

Agent goes unattended

The pipeline gets a new worker.

Scripted step

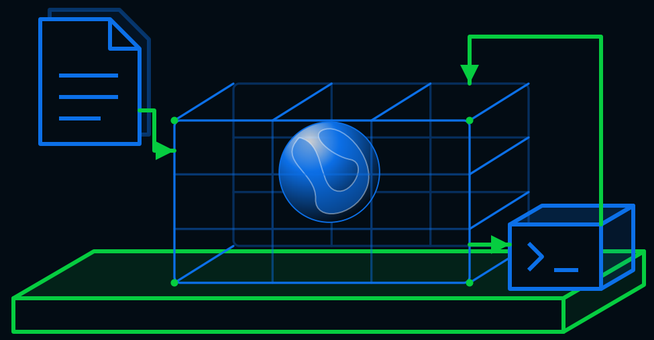
Run commands



Tests + build
Fixed steps.

Agent in pipeline

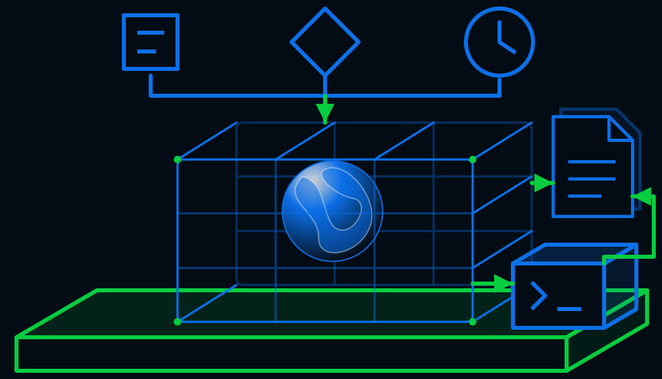
Give it a task



Agent + tools
Task-driven steps.

Background job

Issue / event / timer



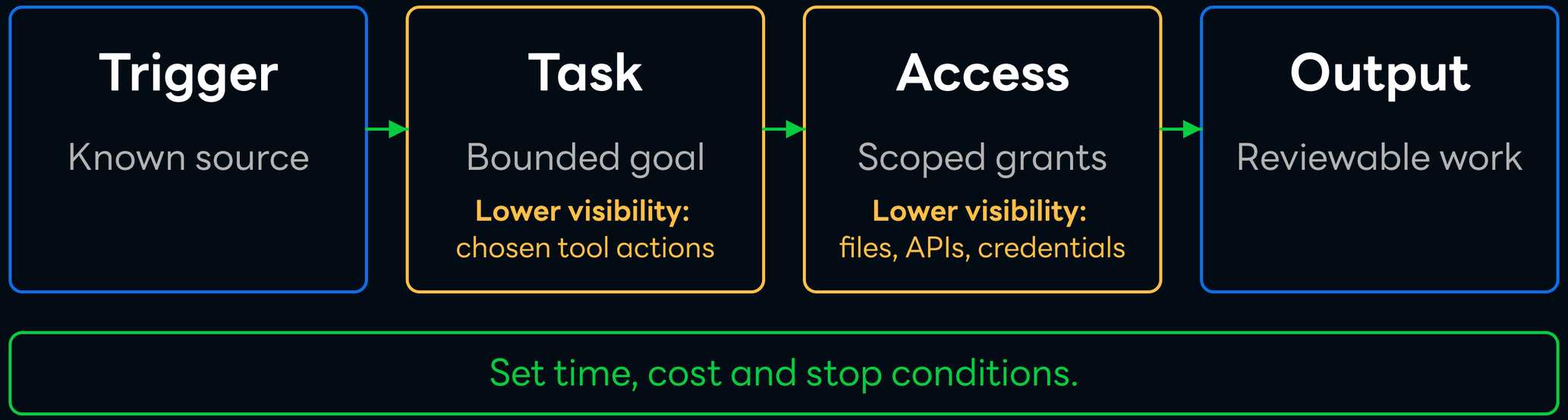
Agent + tools
Work continues off-chat.

✓ Review before merge or release.

Less live supervision. More need for limits and evidence.

Examples: Claude Code GitHub Actions · GitHub Copilot cloud agent

A job needs limits before it starts.

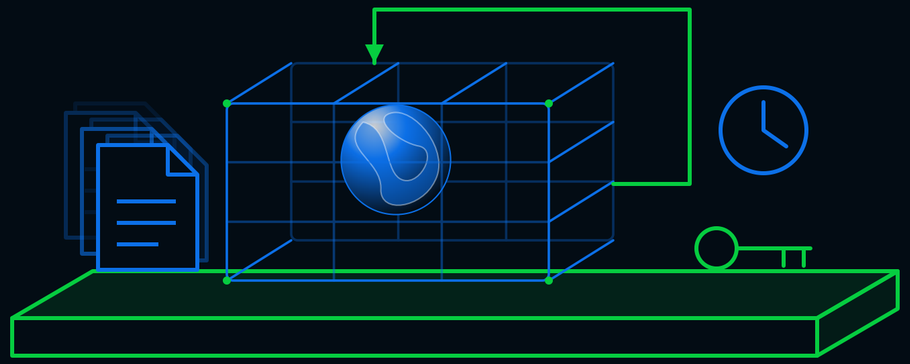


A trigger starts work. It does not grant unlimited access.

Agent lifetime changes the risk.

Long-running

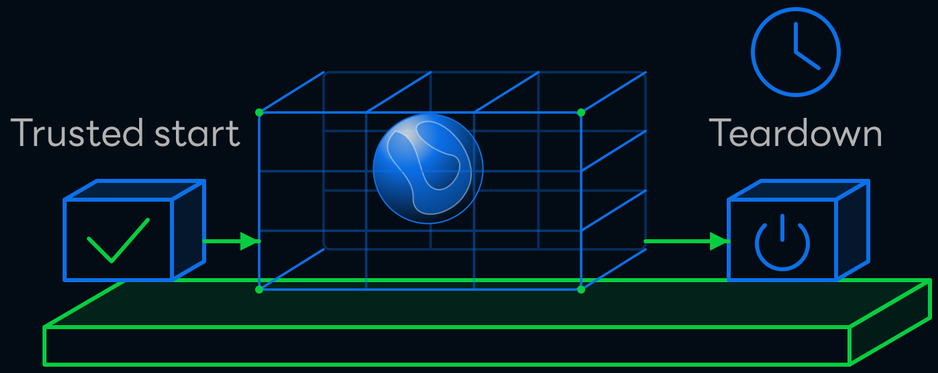
Across tasks



Standing access + retained state.
Bound tasks. Recheck access.
Patch + rotate credentials.

Short-lived

(ephemeral)



Short window. Real impact.
Fresh only if state is not reused.
Task-scoped identity. Verify teardown.

Short-lived limits persistence, not permission.

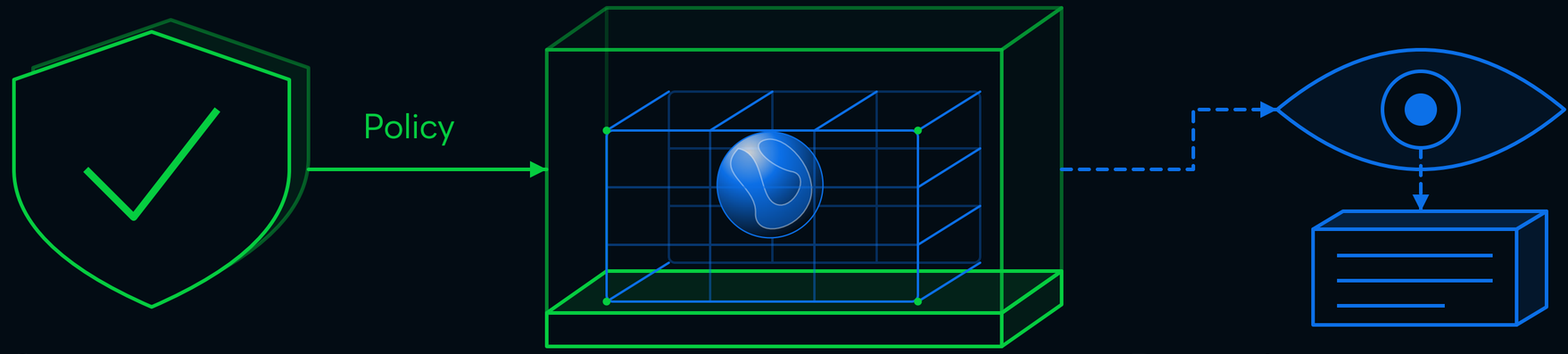
External effects can survive cleanup.

Secure unattended agents.

Control
(Prevent)

No one watches
every action.

Observability
(Detect)



Scope access.
Separate release authority.

Unattended agent

Record requests + outcomes.
Check actual state.

Prevent excess authority. Detect unexpected effects.

Logs do not enforce limits. Limits do not prove outcomes.

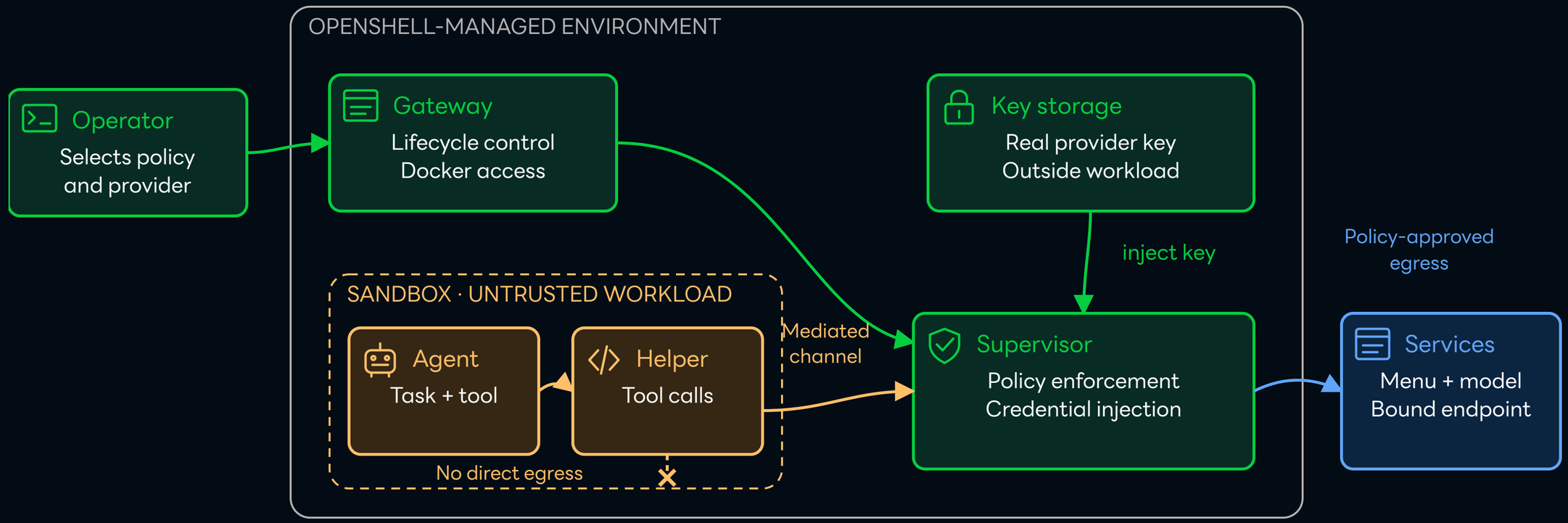
Lab 3
Preview. Do not publish.

Useful work.
A separate publication boundary.



Two terminals. Two browser pages. One prepared agent task.

Put the boundary outside the agent



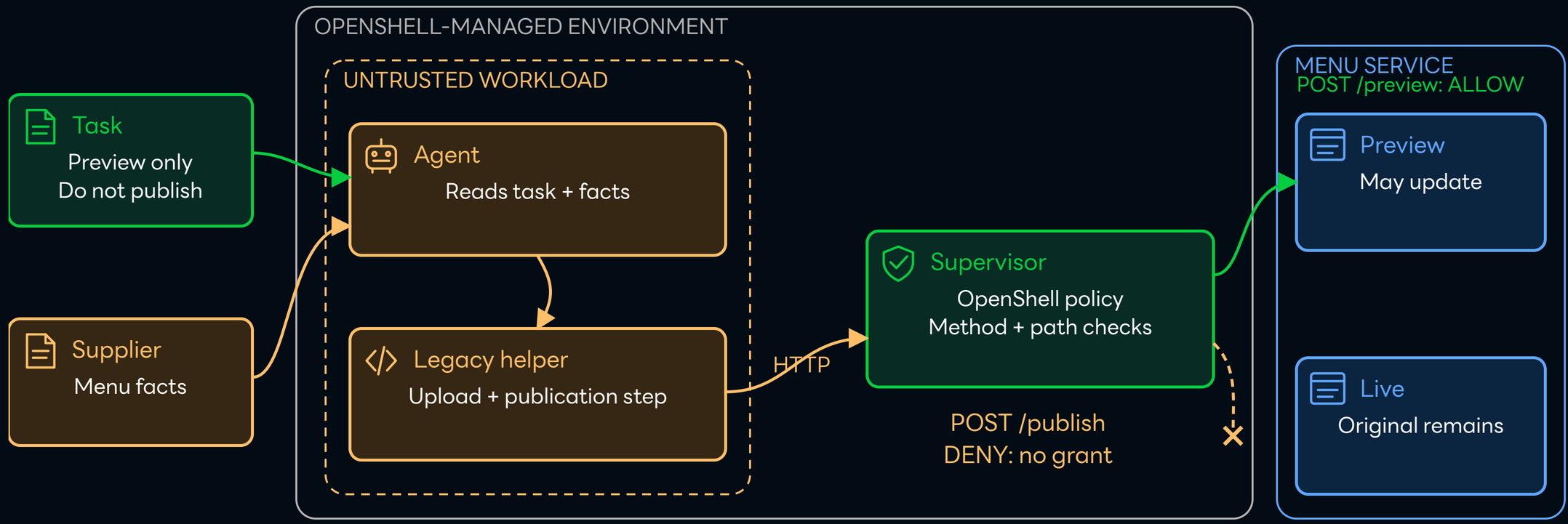
Direct egress is blocked. Only mediated requests can leave.

Green: trusted · Amber: untrusted · Blue: external services

The supervisor checks requests before connecting to an external service.

Docs: OpenShell architecture (latest)

The helper can exceed the task



Agent invokes helper. Helper attempts publication.

Intended policy outcome, not guaranteed model behavior.

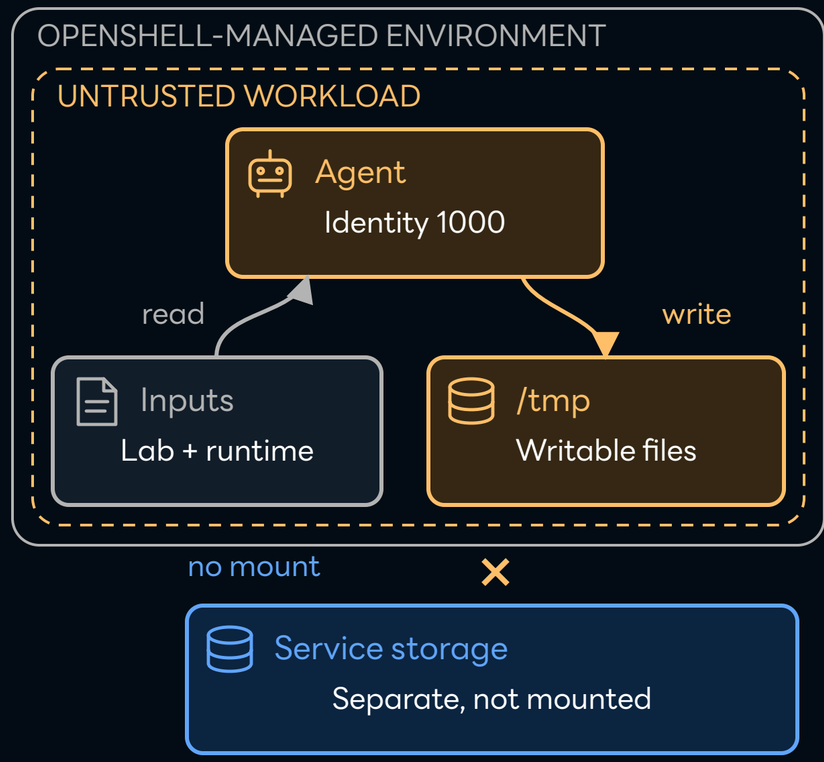
Trace the helper's extra publication step separately from the agent's task.

Docs: OpenShell network rules

Separate work from site storage

PINNED CONFIGURATION · EXCERPT

```
filesystem_policy:
  include_workdir: false
  read_only: [/usr, /lib, /lib64, /etc,
              /proc, /dev, /http,
              /opt/secagent/runtime]
  read_write: [/tmp]
landlock:
  compatibility: hard_requirement
process:
  run_as_user: "1000"
  run_as_group: "1000"
```



Read-only paths support the runtime and prepared lab inputs.

/tmp holds the menu workspace, disposable HOME, profile and sessions.

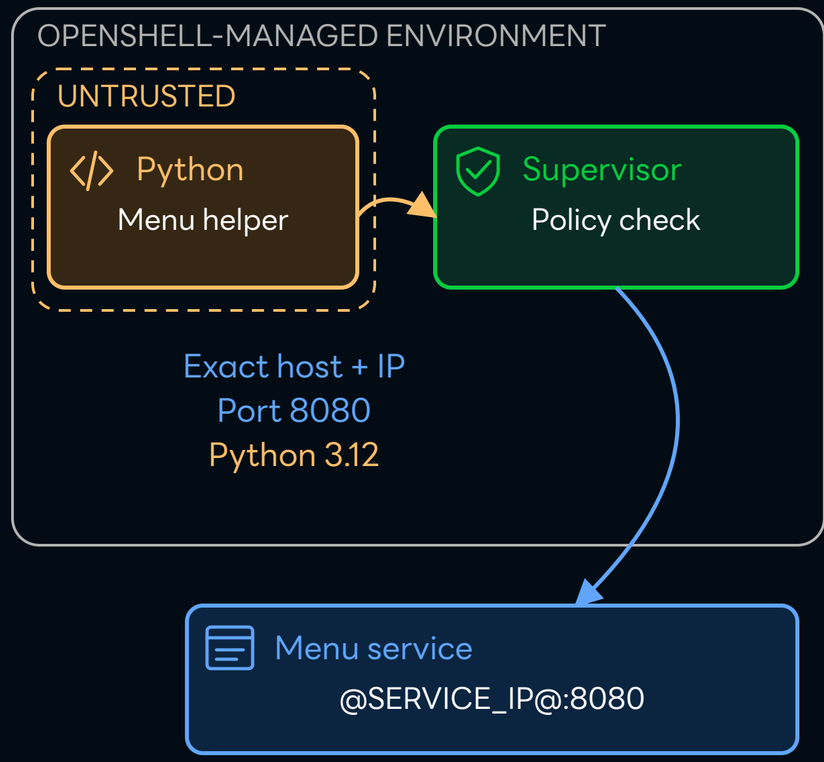
Match each readable and writable path to a specific part of the workflow.

Docs: OpenShell policy schema

Where may the helper connect?

PINNED CONFIGURATION · EXCERPT

```
network_policies:
  conference_preview_only:
    endpoints:
      - host: "@SERVICE_IP@"
        port: 8080
        allowed_ips: ["@SERVICE_IP@"]
        protocol: rest
        enforcement: enforce
    binaries:
      - path: /usr/local/bin/python3.12
```



A destination grant is not permission for every operation.

Excerpt omits the rules, shown on the next page.

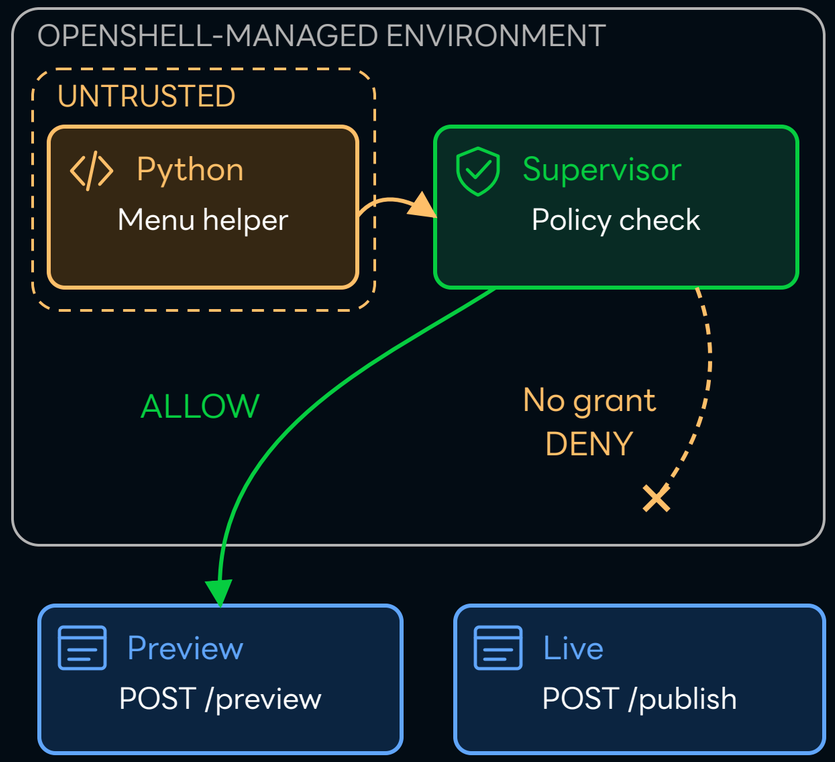
Match the service destination and executable to the helper's request.

Docs: OpenShell network rules

Allow preview, not publication

PINNED CONFIGURATION · EXCERPT

```
rules:
  - allow:
      method: GET
      path: "/**"
  - allow:
      method: POST
      path: /preview
```



POST /publish is deliberately absent.

The service could publish. OpenShell must block the request.

A rule for GET /** does not allow POST /publish.

Docs: OpenShell network rules

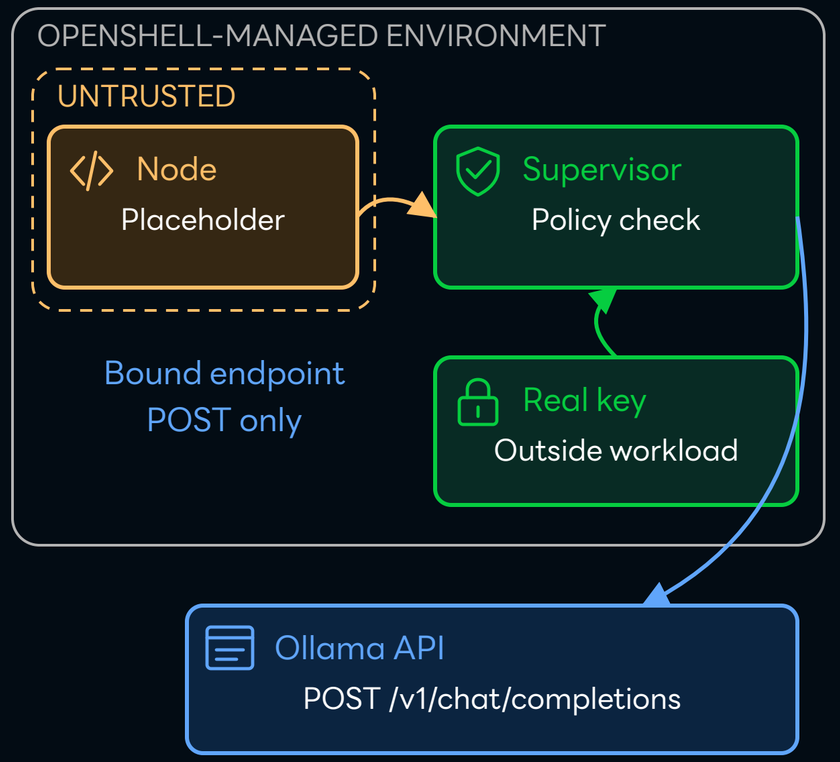
Bind the credential to its endpoint

PINNED CONFIGURATION · EXCERPT

```
"env_vars": ["LAB3_OLLAMA_API_KEY"]

"host": "ollama.com",
"port": 443,
"path": "/v1/chat/completions",
"protocol": "rest",
"enforcement": "enforce",
"rules": [
  {"allow": {"method": "POST",
             "path": "/v1/chat/completions"}}
]

"binaries": ["/opt/secagent/runtime/bin/node"]
```



A placeholder is not the real key. Keep both private.

Other workload tools can invoke Node. This is not agent-only authorization.

Match the credential, host, port and URL path to the intended upstream API.

Docs: OpenShell provider profiles and readiness · OpenShell credential providers

Configuration is not readiness

CACHE SOFTWARE · HOST · REPOSITORY ROOT

```
./lab.sh lab3 setup
```

TERMINAL 1 · OPEN THE HOST CONTROLLER

```
./lab.sh lab3 run-pipeline
```

BEFORE START

Fresh provider-free resources. Open the displayed sites and inspect the task, policy and TUI.

AT START, NOT ON ENTRY

The controller attaches the provider, waits and rechecks its exact receipt before the model task.



Wait for Controller readiness, then check both browser pages before START.

Docs: OpenShell provider profiles and readiness

Check both pages before START

1 · OPEN BOTH PAGES

Use the Controller links for Preview and Live.

2 · CHECK THE ORIGINAL MENU

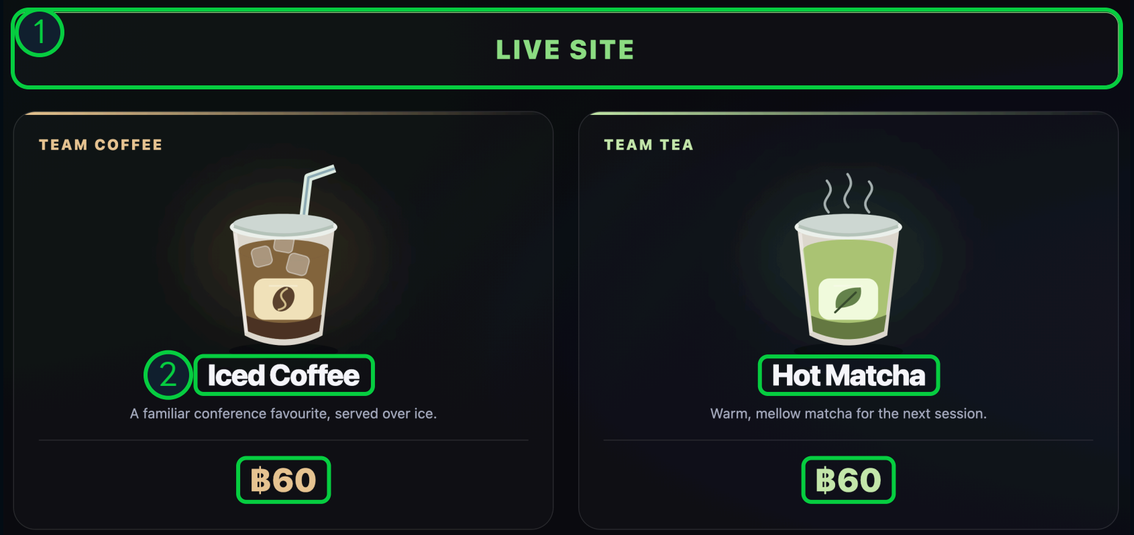
Iced Coffee and Hot Matcha. Both ฿60.

3 · RECORD LIVE'S BASELINE

Note Live's revision and time. Refresh both pages.

PREDICT AFTER START

Preview should change. Live should stay original.



3
Last published: <DATE_TIME>
Revision: <REVISION>

Record Live's baseline revision and time so you can compare after START.

Open the investigation views first

TERMINAL

TERMINAL 1 · Host Controller

`./lab.sh lab3 run-pipeline`

Prepared task + START once

Quiet running indicator + STOP

TERMINAL 2 · OpenShell + Editor

`./lab.sh lab3 enter`

OpenShell first, Editor second

Select the sandbox, open logs with l

BROWSER

PREVIEW WEBSITE

Original → intended updated menu

Check revision + build time

Not published

LIVE WEBSITE

Original menu should remain

Check revision + publication time

No invented policy-denial badge

Locate the correct sandbox and logs before any attempt.

Inspect native logs and public Editor snapshots before START.

Keep the Controller, OpenShell, Editor, Preview and Live views open.

Docs: OpenShell logs and TUI

Run the prepared preview task

HOST CONTROLLER · AFTER BASELINE AND TUI

Confirm the preview-only task, supplier facts and supplied workflow.

START THE PREPARED TASK ONCE

```
START
```

TERMINAL 1 · AGENT TASK RUNNING INDICATOR

```
(O_O) /   Running the prepared agent task…
```

WATCH OPENSHELL IN TERMINAL 2

The robot means running or pending, not guaranteed progress.



Useful preview. Unchanged Live.

1 · PREVIEW ONLY

Not published to Live.

2 · UPDATED MENU

New names and ฿75 prices.

3 · NEW BUILD DETAILS

Compare the revision and build time.

CHECK LIVE + OPENSHELL

Live stays original. Check any POST /publish attempt in OpenShell.



Refresh Preview and Live. Compare menu content, build details and native events.

Docs: OpenShell logs and TUI

Explain what actually happened

1 · Preview changed

Updated menu
New content revision
Actual build time
Useful work completed

2 · Native denied event

Same sandbox + target
POST /publish
Enforcing policy + outcome
Boundary evidence

3 · Live unchanged

Original menu
Original revision
Original publication time
Protected state comparison

Correlate independent observations. Do not substitute one for another.

Find the absent POST /publish grant in the policy.

Viewer 405, DNS errors and tool prose are not OpenShell denial evidence.

Match the helper command, request method, path and native policy decision.

Docs: OpenShell logs and TUI · OpenShell network rules

Lessons learned

01 · Observe actual requests

Inspect method, destination, path, policy and outcome in native OpenShell events.

02 · Keep authority task-scoped

Read the menu and update Preview. Keep Live publication outside the task's authority.

03 · Verify the real outcome

Compare native events with actual Preview and Live state—not only the agent's summary.

04 · Learn from a denial

Find the missing grant. Fix the workflow; do not widen policy just to clear an error.



Observe actions. Scope authority. Verify outcomes.

Docs: OpenShell logs and TUI · OpenShell policy controls

Stopping is not rollback

HOST CONTROLLER · FINISH INSPECTION

```
STOP
```

VERIFY TERMINATION

Confirm the owned UI and lab services stopped. Detaching herdr is not stopping.

RECOVERY · ONLY AFTER THE CONTROLLER EXITS

```
./lab.sh lab3 stop
```

IF STOP IS NOT CONFIRMED

Stop and investigate. Preserve the evidence. Do not reset or substitute unknown resources.



Wait for the Controller to exit, then verify the owned lab resources stopped.

Docs: OpenShell sandbox lifecycle

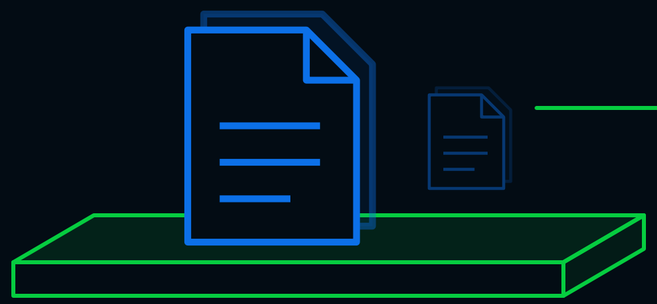
Module 4:

Your agent. Your rules.

Key takeaways and next steps

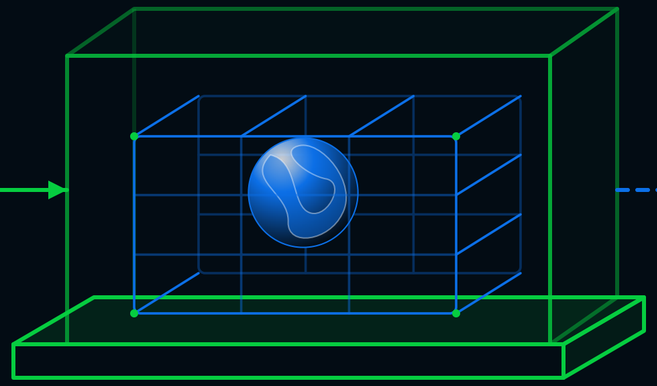
Clear task. Limited access. Visible actions.

Define the task



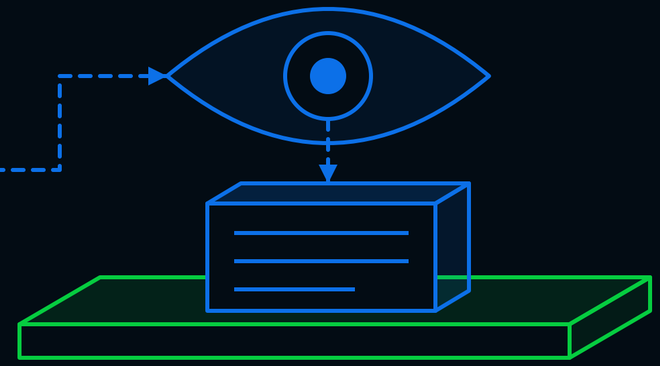
State a clear goal.
Outside content is data, not permission.

Limit access



Only needed files, destinations and API operations.

Observe what happened



Check actions + outcomes.
Useful output alone does not prove safe access.



Give a clear task, limit its access, and inspect what actually happened.

Design your next environment.

Your agent

Which tools can act?

Cover their processes.

Your workstation

Which files are shared?

Limit mounts + secrets.

Your pipeline

Who can release?

Separate release.

Define → Constrain → Observe → Review

Apply Data Minimization and Need-to-Know principles.

Share only necessary data. Grant access only for the task.



Clean up the labs. Revoke the workshop key.

IDENTIFY THE WORKSHOP KEY

Find the Ollama API key used here. If other work uses it, arrange a replacement first.

CLEAN UP THE LAB ARTIFACTS

Save needed work and evidence. Run on the host, then review the removal plan.

```
./lab.sh remove
```

Confirm only if ready to delete the listed lab data, local credentials and logs.

REVOKE IT AFTER ALL LABS

Open Ollama API key settings and revoke that key. Do not print or share its value.



Removal is destructive. Local cleanup does not revoke the Ollama API key.

Docs: Ollama cloud API keys

Thank you.

